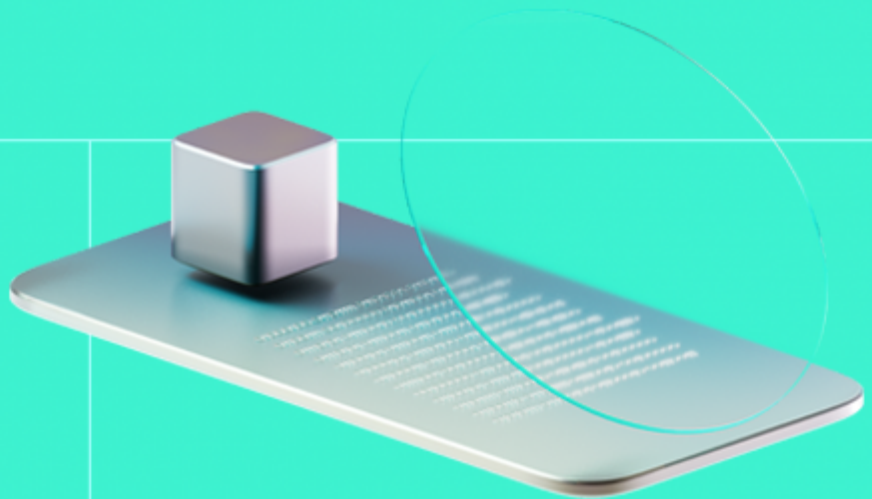




Smart Contract Code Review And Security Analysis Report

Customer: Skale

Date: 27/01/2026



We express our gratitude to the Skale team for the collaborative engagement that enabled the execution of this Smart Contract Security Assessment.

SKALE is ERC777-based fungible token contracts (SkaleToken and SkaleTokenL2) with role-based access control and integration points for Optimism L2 bridging and on-chain staking/delegation systems.

Document

Name	Smart Contract Code Review and Security Analysis Report for Skale
Audited By	Seher Saylik, David Camps
Approved By	Ivan Bondar
Website	https://www.skale.space/
Changelog	19/01/2025 - Preliminary Report
	27/01/2025 - Final Report
Platform	Base
Language	Solidity
Tags	ERC777, Layer 2, Fungible Token
Methodology	https://docs.hacken.io/methodologies/smart-contracts

Review

Scope

Repository	https://github.com/skalenetwork/skale-manager/blob/develop/contracts/I2/SkaleTokenL2.sol
Commit	bed9af784ddeb99cc05c023d2b13b6729c9f3a06
Retest	88acab0ceaff046892810af5a336819191cc2348

Audit Summary

The system users should acknowledge all the risks summed up in the risks section of the report

3	0	3	0
Total Findings	Resolved	Accepted	Mitigated

Findings by Severity

Severity	Count
Critical	0
High	0
Medium	0
Low	0

Vulnerability	Severity Status	
F-2026-14782 - Incorrect ERC721 Naming and NatSpec Documentation in Fungible Token Contracts	Info	Accepted
F-2026-14787 - Redundant SafeMath Usage in Solidity 0.8.0+ Contracts	Info	Accepted
F-2026-14790 - Floating Pragma	Info	Accepted

Documentation quality

- Functional requirements are partially missed.
- Technical description is provided.

Code quality

- The code implements confusing naming.
- The development environment is configured.

Test coverage

Code coverage of the project is **39.5%** (branch coverage).

- Extensive tests were provided for the SkaleToken contract; however, no tests were supplied for the ERC777 and SkaleTokenL2 contracts.

Table of Contents

System Overview	6
Privileged Roles	6
Potential Risks	
Findings	8
Vulnerability Details	8
Disclaimers	14
Appendix 1. Definitions	15
Severities	15
Potential Risks	15
Appendix 2. Scope	16
Appendix 3. Additional Valuables	17

System Overview

SKALE is an ERC777-based token system with the following contracts:

SkaleToken — primary SKALE Network token implementing ERC777 (with ERC20 backwards compatibility). It integrates with external locker contracts to enforce token locks during delegation/staking.

It has the following attributes:

- Name: SKALE
- Symbol: SKL
- Decimals: 18
- Max supply: 7,000,000,000 tokens.

SkaleTokenL2 — L2-compatible version of SkaleToken designed for Optimism. It allows the Optimism bridge to mint and burn tokens during cross-chain transfers.

ERC777 — a modified OpenZeppelin ERC777 implementation providing the core token functionality.

Permissions — an upgradeable access control base contract.

Privileged roles

- **DEFAULT_ADMIN_ROLE** — Deployer/owner with full administrative control; can grant and revoke all roles.
- **MINTER_ROLE** — Authorized to mint tokens up to the cap. Granted to SkaleManager (L1) or the Optimism bridge (L2).
- **Default Operators** — Can transfer tokens on behalf of any holder unless individually revoked via `revokeOperator()`.

Potential Risks

- The contracts in scope (SkaleToken, SkaleTokenL2, ERC777, Permissions) make critical external calls to out-of-scope contracts including ContractManager, TokenState, DelegationController, Locker and Punisher, whose security was not verified as part of this audit. Vulnerabilities, misconfigurations, or malicious behavior in these external dependencies could compromise the security of the in-scope contracts regardless of the findings in this report.
- Every token transfer depends on a chain of external calls (ContractManager → TokenState → DelegationController → Punisher), and if any contract in this chain reverts or becomes unavailable, all token transfers will be blocked.
- Default operators set at deployment have the ability to transfer tokens on behalf of any holder without explicit approval. Users must actively call `revokeOperator()` to opt out. If a default operator address is compromised, it could move tokens from any user who has not explicitly revoked it.

Findings

Vulnerability Details

[F-2026-14782](#) - Incorrect ERC721 Naming and NatSpec Documentation in Fungible Token Contracts - Info

Description:

The `_beforeTokenTransfer()` hook in both `ERC777.sol` and `SkaleToken.sol` uses incorrect ERC721 (NFT) terminology for a fungible ERC777 token. The parameter is named `tokenId` when it actually represents `amount` - the quantity of fungible tokens being transferred. Additionally, the NatSpec documentation in `ERC777.sol` describes NFT behavior that is semantically impossible for a fungible token, creating misleading documentation that could confuse developers, auditors, and integrators.

ERC777.sol :

The NatSpec comments describe NFT behavior that does not apply to fungible tokens:

```
/**
 * @dev Hook that is called before any token transfer. This includes
 * calls to {send}, {transfer}, {operatorSend}, minting and burning.
 *
 * Calling conditions:
 *
 * - when `from` and `to` are both non-zero, ``from``'s `tokenId` will be
 *   transferred to `to`.
 * - when `from` is zero, `tokenId` will be minted for `to`.
 * - when `to` is zero, ``from``'s `tokenId` will be burned.
 * - `from` and `to` are never both zero.
 *
 * To learn more about hooks, head to xref:ROOT:extending-contracts.adoc#using-hooks[Using Hooks].
 */
function _beforeTokenTransfer(address operator, address from, address to, uint256 tokenId) internal virtual { }
```

SkaleToken.sol:

The fourth parameter is named `tokenId` but represents amount.

```
function _beforeTokenTransfer(
    address, // operator
```



```

    address from,
    address, // to
    uint256 tokenId)
    internal override
    {
        uint256 locked = getAndUpdateLockedAmount(from);
        if (locked > 0) {
            require(
                balanceOf(from) >= locked.add(tokenId),
                "Token should be unlocked for transferring"
            );
        }
    }
}

```

Assets:

- contracts/SkaleToken.sol [<https://github.com/skalenetwork/skale-manager>]
- contracts/thirdparty/openzeppelin/ERC777.sol [<https://github.com/skalenetwork/skale-manager>]

Status:

Accepted

Classification

Impact Rate:	1/5
Likelihood Rate:	5/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation: Update both the parameter name and NatSpec documentation to accurately reflect fungible token semantics.

Resolution: The issue has been addressed across the codebase in commit [88acab0](#). However, the ERC777 contract remains unchanged. The client clarified that [ERC777.sol](#) is an external OpenZeppelin implementation and is intentionally kept unmodified to preserve upstream compatibility and integrity.

[F-2026-14787](#) - Redundant SafeMath Usage in Solidity 0.8.0+

Contracts - Info

Description:

The contracts import and use OpenZeppelin's SafeMath library for arithmetic operations despite being compiled with Solidity version 0.8.17, which has native overflow and underflow protection built into the compiler. This results in redundant checks that waste gas without providing any additional security benefit.

Starting from Solidity 0.8.0, arithmetic operations revert on overflow and underflow by default. The SafeMath library was essential for Solidity versions prior to 0.8.0, but is now obsolete and adds unnecessary overhead.

For instance, SkaleToken.sol::

```
pragma solidity 0.8.17;
import { SafeMath } from "@openzeppelin/contracts/utils/math/SafeMath.sol";
// ...
contract SkaleToken is ERC777, Permissions, ReentrancyGuard, IDelegatableToken, IMintableToken {
    using SafeMath for uint;
    // ...
}
```

Assets:

- contracts/I2/SkaleTokenL2.sol [<https://github.com/skalenetwork/skale-manager>]
- contracts/SkaleToken.sol [<https://github.com/skalenetwork/skale-manager>]
- contracts/thirdparty/openzeppelin/ERC777.sol [<https://github.com/skalenetwork/skale-manager>]

Status:

Accepted

Classification

Impact Rate: 1/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Replace all SafeMath operations with native Solidity operators.

Resolution: The Skale team replaced SafeMath operations with native Solidity operators in commit `f40df28`. However, the ERC777 contract continues to use SafeMath, as the client stated that it is an external OpenZeppelin implementation and is intentionally left unmodified to preserve upstream compatibility and integrity.

F-2026-14790 - Floating Pragma - Info

Description:

In Solidity development, the pragma directive specifies the compiler version to be used, ensuring consistent compilation and reducing the risk of issues caused by version changes. However, using a floating pragma (e.g., `^0.8.xx`) introduces uncertainty, as it allows contracts to be compiled with any version within a specified range. This can result in discrepancies between the compiler used in testing and the one used in deployment, increasing the likelihood of vulnerabilities or unexpected behavior due to changes in compiler versions.

The project currently uses floating pragma declarations (`^0.8.0`, `^0.8.17`) in its Solidity contracts. This increases the risk of deploying with a compiler version different from the one tested, potentially reintroducing known bugs from older versions or causing unexpected behavior with newer versions. These inconsistencies could result in security vulnerabilities, system instability, or financial loss. Locking the pragma version to a specific, tested version is essential to prevent these risks and ensure consistent contract behavior.

Assets:

- contracts/thirdparty/openzeppelin/ERC777.sol [https://github.com/skalenetwork/skale-manager]
- contracts/Permissions.sol [https://github.com/skalenetwork/skale-manager]

Status:

Accepted

Classification

Impact Rate: 1/5

Likelihood Rate: 2/5

Exploitability: Dependent

Complexity: Simple

Severity: Info

Recommendations

Remediation:

It is recommended to **lock the pragma version** to the specific version that was used during development and testing. This ensures that the contract will always be compiled with a known, stable compiler version, preventing unexpected changes in behavior due to compiler updates. For example,

instead of using `^0.8.17`, explicitly define the version with `pragma solidity 0.8.17;`.

Before selecting a version, review known bugs and vulnerabilities associated with each Solidity compiler release. This can be done by referencing the official Solidity compiler release notes: [Solidity GitHub releases](#) or [Solidity Bugs by Version](#). Choose a compiler version with a good track record for stability and security.

Resolution:

The risk of the given finding is accepted, and the client stated that it will be fixed in one of their future releases.

Disclaimers

Hacken Disclaimer

The smart contracts given for audit have been analyzed based on best industry practices at the time of the writing of this report, with cybersecurity vulnerabilities and issues in smart contract source code, the details of which are disclosed in this report (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

The report contains no statements or warranties on the identification of all vulnerabilities and security of the code. The report covers the code submitted and reviewed, so it may not be relevant after any modifications. Do not consider this report as a final and sufficient assessment regarding the utility and safety of the code, bug-free status, or any other contract statements.

While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only — we recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contracts.

English is the original language of the report. The Consultant is not responsible for the correctness of the translated versions.

As part of Hacken's ongoing quality assurance process, we may conduct re-audits of select projects. These re-audits are performed independently from the original audit and are intended solely for internal quality control and improvement. Updated reports resulting from such re-audits will be shared privately with the respective clients and may be published on the Hacken website only with their explicit consent.

The sole authoritative source for finalized and most up-to-date versions of all reports remains the Audits section at <https://hacken.io/audits/>.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have vulnerabilities that can lead to hacks. Thus, the Consultant cannot guarantee the explicit security of the audited smart contracts.

Appendix 1. Definitions

Severities

When auditing smart contracts, Hacken is using a risk-based approach that considers **Likelihood**, **Impact**, **Exploitability** and **Complexity** metrics to evaluate findings and score severities.

Reference on how risk scoring is done is available through the repository in our Github organization:

[hknio/severity-formula](https://github.com/hacken/severity-formula)

Severity	Description
Critical	Critical vulnerabilities are usually straightforward to exploit and can lead to the loss of user funds or contract state manipulation.
High	High vulnerabilities are usually harder to exploit, requiring specific conditions, or have a more limited scope, but can still lead to the loss of user funds or contract state manipulation.
Medium	Medium vulnerabilities are usually limited to state manipulations and, in most cases, cannot lead to asset loss. Contradictions and requirements violations. Major deviations from best practices are also in this category.
Low	Major deviations from best practices or major Gas inefficiency. These issues will not have a significant impact on code execution.

Potential Risks

The "Potential Risks" section identifies issues that are not direct security vulnerabilities but could still affect the project's performance, reliability, or user trust. These risks arise from design choices, architectural decisions, or operational practices that, while not immediately exploitable, may lead to problems under certain conditions. Additionally, potential risks can impact the quality of the audit itself, as they may involve external factors or components beyond the scope of the audit, leading to incomplete assessments or oversight of key areas. This section aims to provide a broader perspective on factors that could affect the project's long-term security, functionality, and the comprehensiveness of the audit findings.

Appendix 2. Scope

The scope of the project includes the following smart contracts from the provided repository:

Scope Details	
Repository	https://github.com/skalenetwork/skale-manager/blob/develop/contracts/I2/SkaleTokenL2.sol
Commit	bed9af784ddeb99cc05c023d2b13b6729c9f3a06
Retest	88acab0ceaff046892810af5a336819191cc2348
Whitepaper	https://github.com/skalenetwork/skale-manager/tree/develop/docs
Requirements	N/A
Technical Requirements	https://github.com/skalenetwork/skale-manager/tree/develop/docs

Asset	Type
contracts/I2/SkaleTokenL2.sol [https://github.com/skalenetwork/skale-manager]	Smart Contract
contracts/Permissions.sol [https://github.com/skalenetwork/skale-manager]	Smart Contract
contracts/SkaleToken.sol [https://github.com/skalenetwork/skale-manager]	Smart Contract
contracts/thirdparty/openzeppelin/ERC777.sol [https://github.com/skalenetwork/skale-manager]	Smart Contract

Appendix 3. Additional Valuables

Additional Recommendations

The smart contracts in the scope of this audit could benefit from the introduction of automatic emergency actions for critical activities, such as unauthorized operations like ownership changes or proxy upgrades, as well as unexpected fund manipulations, including large withdrawals or minting events. Adding such mechanisms would enable the protocol to react automatically to unusual activity, ensuring that the contract remains secure and functions as intended.

To improve functionality, these emergency actions could be designed to trigger under specific conditions, such as:

- Detecting changes to ownership or critical permissions.
- Monitoring large or unexpected transactions and minting events.
- Pausing operations when irregularities are identified.

These enhancements would provide an added layer of security, making the contract more robust and better equipped to handle unexpected situations while maintaining smooth operations.

Frameworks and Methodologies

This security assessment was conducted in alignment with recognised penetration testing standards, methodologies and guidelines, including the [NIST SP 800-115 – Technical Guide to Information Security Testing and Assessment](#), and the [Penetration Testing Execution Standard \(PTES\)](#). These assets provide a structured foundation for planning, executing, and documenting technical evaluations such as vulnerability assessments, exploitation activities, and security code reviews. Hacken's internal penetration testing methodology extends these principles to Web2 and Web3 environments to ensure consistency, repeatability, and verifiable outcomes.